

CYBERSNOVA CYBER SECURITY INTERNSHIP

PROJECT 2

PortSwigger Web Security Academy Labs

Apprentice-Level Lab Assessment — Professional Technical Report

Submitted By	Yashdeep Sankhla
Prepared By	Yashdeep Sankhla
Internship Domain	Cyber Security
Project Title	PortSwigger Web Security Academy Labs
Platform	https://portswigger.net/web-security
Requirement	Complete any five Apprentice-level labs
Primary Labs Reported	SQL Injection, Path Traversal, Reflected XSS, Broken Access Control, CSRF
Assessment Environment	PortSwigger Web Security Academy + Burp Suite

Tools Used: Burp Suite · Web Browser / Burp Browser · Developer Tools

Problem Statement

Project 2: PortSwigger Web Security Academy Labs

Complete any five Apprentice-level labs available on the PortSwigger Web Security Academy. For each completed lab, explain the vulnerability, describe the exploitation process, and recommend appropriate defensive measures. Prepare a detailed report summarizing the learning from each lab.

Resources — Platform

- <https://portswigger.net/web-security>

Submission Requirements

- Screenshots
- Technical Report
- Lab Completion Proof

For this assignment, five Apprentice-level labs were completed and are reported as the primary submission set: SQL Injection (retrieve hidden data), File Path Traversal (simple case), Reflected XSS (HTML context, nothing encoded), Broken Access Control (user role controlled by request parameter), and CSRF (no defenses). Burp Suite was used throughout to intercept, modify, and replay HTTP requests, and each lab was validated using the official PortSwigger completion banner. The remainder of this document presents the assessment methodology, the five lab write-ups with supporting evidence, cross-lab security analysis, learning outcomes, and conclusions produced in response to the above problem statement.

Table of Contents

Problem Statement	2
1. Executive Summary	4
2. Assessment Methodology	5
3.1 SQL Injection Vulnerability in WHERE Clause	6
3.2 File Path Traversal, Simple Case	9
3.3 Reflected XSS into HTML Context with Nothing Encoded	13
3.4 User Role Controlled by Request Parameter	16
3.5 CSRF Vulnerability with No Defenses	20
4. Cross-Lab Security Analysis	23
5. Learning Outcomes	24
6. Conclusion	24
Appendix A — Additional Completed Lab Evidence	25
Appendix B — Evidence Inventory	26
Appendix C — Official PortSwigger References	26

1. Executive Summary

This report documents the practical completion of Apprentice-level Web Security Academy labs from PortSwigger. The evidence collected during each lab was used to establish the testing workflow, exploitation steps, impact, remediation guidance, and lab-completion status. Five representative labs are presented as the primary submission set, satisfying the project requirement of completing any five Apprentice-level labs.

The five primary labs cover multiple vulnerability classes: SQL injection, file path traversal, cross-site scripting, broken access control, and cross-site request forgery. The exercises demonstrate a repeatable web-security workflow: identify an input or trust boundary, capture the request with Burp Suite, manipulate attacker-controlled values, validate the server response, confirm impact, and document the result.

The evidence archive also contains additional completed labs beyond the five required for submission. These are listed in Appendix A as supplementary evidence rather than being treated as required deliverables.

1.1 Project Requirement

The assignment requires completion of any five Apprentice-level PortSwigger Web Security Academy labs. For each completed lab, the submission explains the vulnerability, exploitation process, defensive measures, and learning outcome, supported by screenshots and lab-completion proof.

1.2 Primary Lab Summary

ID	Official Lab	Vulnerability	Proof of Impact	Status
01	SQL injection vulnerability in WHERE clause allowing retrieval of hidden data	SQL Injection	Unreleased products retrieved	Completed
02	File path traversal, simple case	Path Traversal	/etc/passwd retrieved	Completed
03	Reflected XSS into HTML context with nothing encoded	Reflected XSS	JavaScript execution demonstrated	Completed
04	User role controlled by request parameter	Broken Access Control	Admin access obtained and Carlos deleted	Completed
05	CSRF vulnerability with no defenses	CSRF	Victim email-change action triggered	Completed

2. Assessment Methodology

Reconnaissance

Reviewed the lab application and identified normal application behavior and candidate inputs.

Traffic Interception

Used Burp Suite Proxy/HTTP history to capture application requests and understand request parameters.

Input Manipulation

Modified relevant parameters, cookies, paths, or form data to test trust boundaries.

Validation

Used Burp Repeater and the browser to compare responses and verify whether the intended behavior could be influenced.

Exploitation

Applied the lab-specific proof-of-concept technique required to demonstrate the vulnerability.

Impact Confirmation

Verified the expected lab objective, such as retrieving hidden data, gaining administrative access, or changing an account property.

Evidence Collection

Captured screenshots of baseline behavior, manipulated requests/responses, successful exploitation, and lab completion.

Remediation Analysis

Mapped each finding to practical secure-development controls.

2.1 Tools and Technologies

- PortSwigger Web Security Academy — deliberately vulnerable training applications.
- Burp Suite — HTTP interception, request modification, Repeater analysis, and evidence capture.
- Web browser / Burp browser — application interaction and execution of client-side proof-of-concept payloads.
- Developer tools / page source inspection — used where client-side code or DOM behavior was relevant.

3.1 SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

Vulnerability	Difficulty	Risk Context	Lab Status
SQL Injection (SQLi)	Apprentice	High — unauthorized database query manipulation can expose data that the application intended to hide.	Completed

3.1.1 Lab Problem Statement

Official lab objective: The lab contains a SQL injection vulnerability in the product category filter. The objective is to perform SQL injection that causes the application to display one or more unreleased products.

Official PortSwigger lab page: <https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>

3.1.2 Vulnerability Overview

SQL injection occurs when untrusted input is concatenated into a database query without safe parameterization. In this lab, the category value is incorporated into a WHERE clause that also contains a condition restricting results to released products.

3.1.3 Assessment Objective

Demonstrate that a user-controlled product-category parameter can alter the backend SQL logic and return unreleased products.

3.1.4 Exploitation Procedure

1. Open the shopping application and observe normal category-filter behavior.
2. Capture the category request in Burp Suite and identify the category parameter as an attacker-controlled input.
3. Send the request to Burp Repeater and test how SQL metacharacters affect the response.
4. Modify the category value using the lab proof of concept: '+OR+1=1--'. The injected boolean condition makes the WHERE clause broadly true and the comment sequence removes the remaining query logic.
5. Forward the manipulated request and inspect the response. Previously hidden/unreleased products are returned.
6. Confirm the Academy banner reports the lab as solved and preserve the completion screenshot.

3.1.5 Impact / Security Significance

- Unauthorized disclosure of database-backed records.
- Potential exposure of unreleased products, internal records, or other information controlled by query conditions.

- Depending on the application and database privileges, SQL injection can lead to authentication bypass, data modification, or broader database compromise.

3.1.6 Defensive Measures

- Use parameterized queries / prepared statements for all database access.
- Do not construct SQL statements by concatenating raw user input.
- Apply allow-list validation where input is expected to be a constrained value such as a category identifier.
- Use a database account with least privilege so a compromised query cannot perform unnecessary operations.
- Monitor and test query interfaces with SAST/DAST and security regression tests.

3.1.7 Learning Outcome

The lab reinforced that SQL injection is fundamentally a trust-boundary failure between application input and database query construction. Burp Repeater made the cause-and-effect relationship visible by allowing the request to be modified and compared directly.

Suggested classification: CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)

3.1.8 Evidence

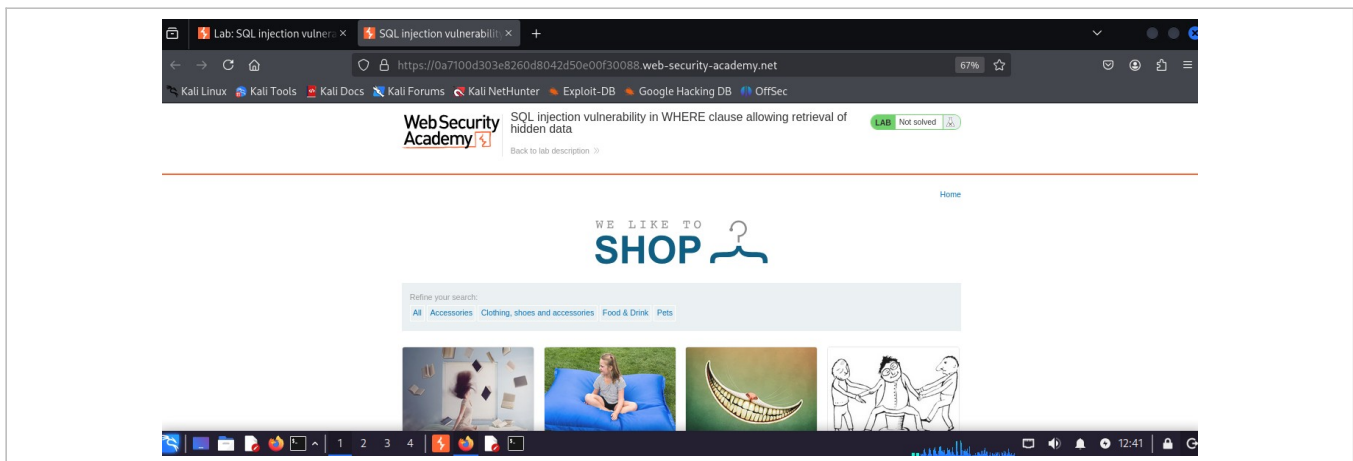


Figure 1 — Baseline shopping application before manipulation.

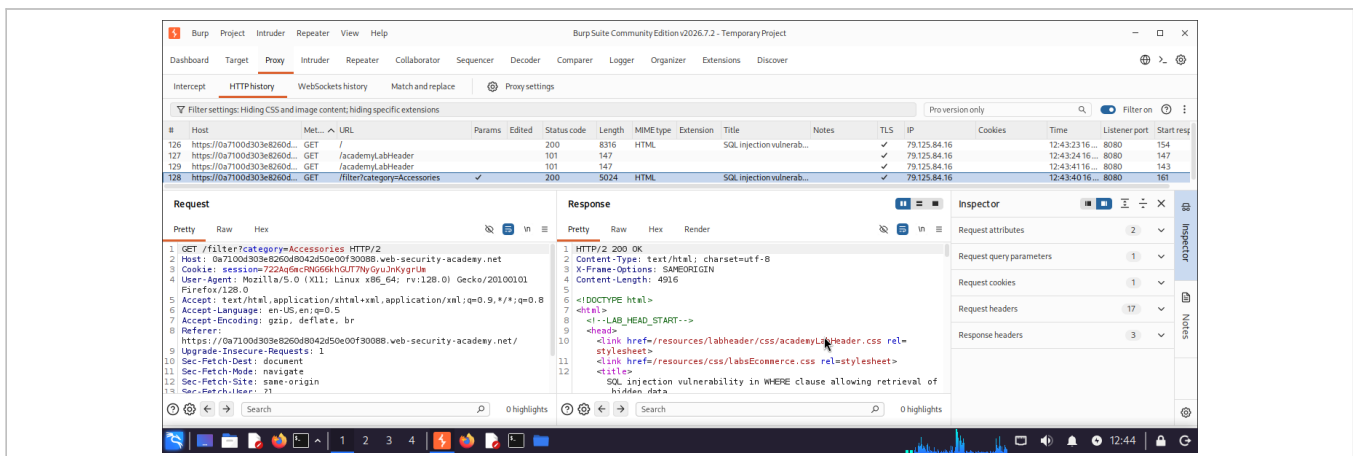


Figure 2 — Normal category request captured in Burp Suite.

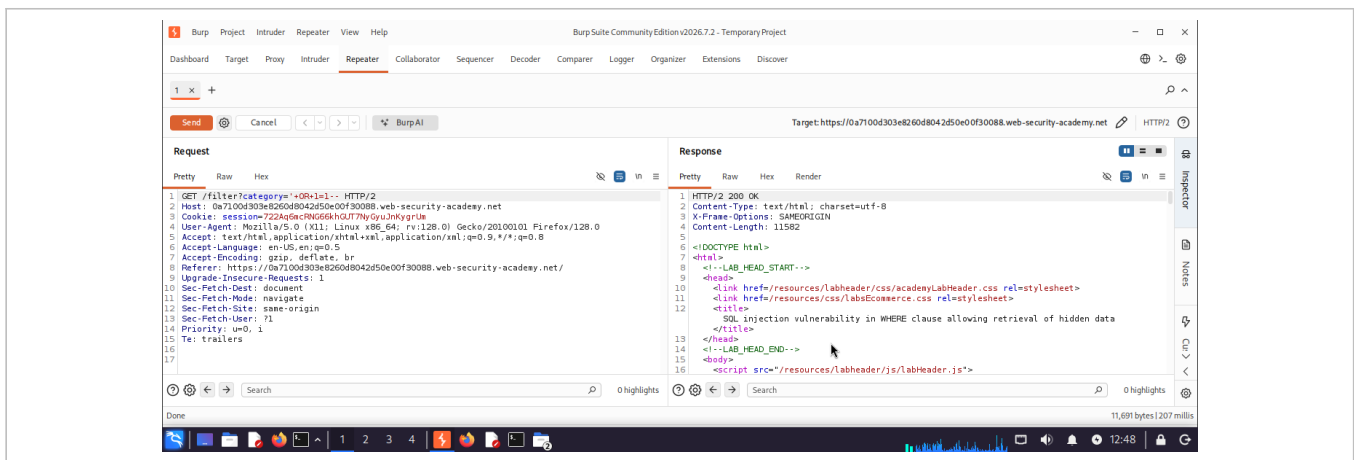


Figure 3 — SQL injection payload inserted into the category parameter.

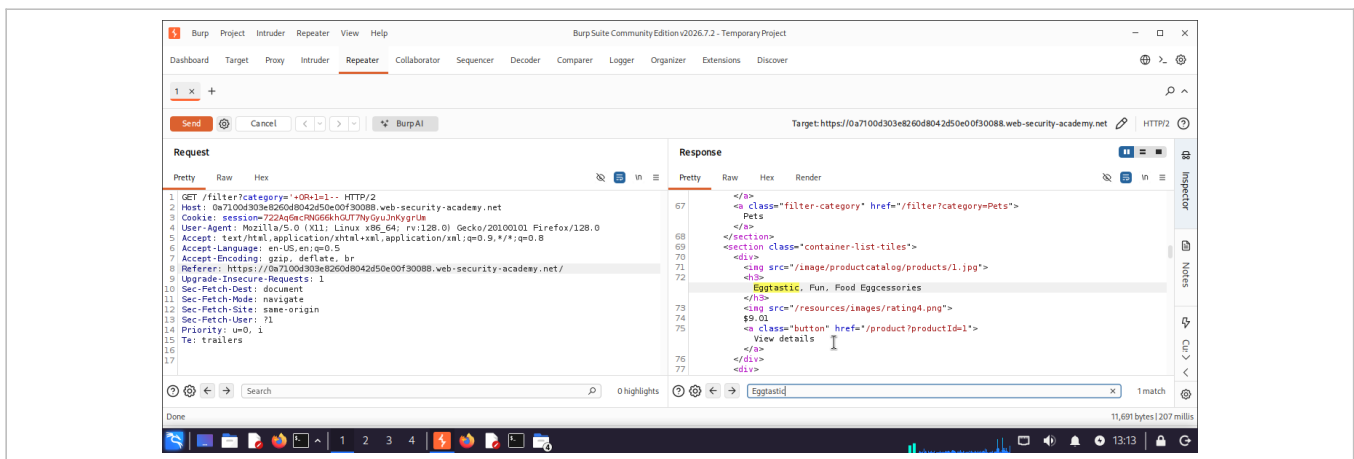


Figure 4 — Response showing previously hidden/unreleased products.

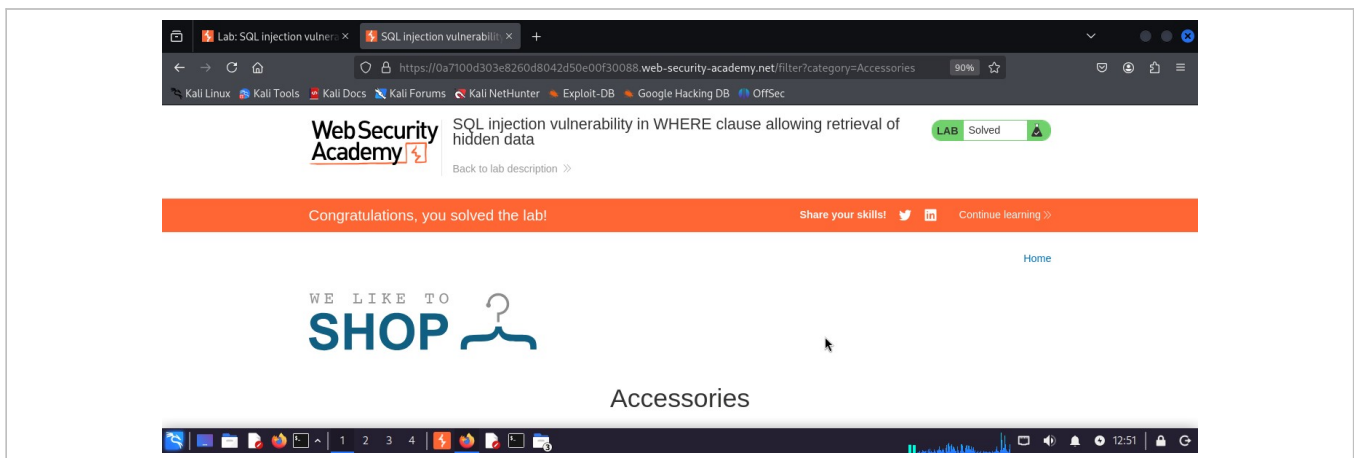


Figure 5 — PortSwigger lab completion proof.

3.2 File Path Traversal, Simple Case

Vulnerability	Difficulty	Risk Context	Lab Status
Path Traversal / Directory Traversal	Apprentice	High — arbitrary file read can expose operating-system, configuration, credential, or application data.	Completed

3.2.1 Lab Problem Statement

Official lab objective: This lab contains a path traversal vulnerability in the display of product images. To solve the lab, retrieve the contents of the `/etc/passwd` file.

Official PortSwigger lab page: <https://portswigger.net/web-security/file-path-traversal/lab-simple>

3.2.2 Vulnerability Overview

Path traversal occurs when an application uses a user-controlled file path without correctly constraining it to an intended directory. Sequences such as `../` can move the path outside the expected directory and allow access to arbitrary files readable by the application process.

3.2.3 Assessment Objective

Retrieve the contents of the Linux `/etc/passwd` file by manipulating the filename used to load a product image.

3.2.4 Exploitation Procedure

1. Browse a product and identify an image request such as `GET /image?filename=<image>.jpg`.
2. Capture the image request in Burp Suite and confirm that the filename parameter is user-controlled.
3. Replace the normal filename with an absolute path and observe that it is not resolved as intended.
4. Use relative traversal sequences to move upward through the directory hierarchy using repeated `../` segments.
5. Send the modified request and inspect the response in Burp. The response contains the contents of `/etc/passwd`.
6. Verify the PortSwigger lab status changes to solved and capture the completion proof.

3.2.5 Impact / Security Significance

- Unauthorized disclosure of operating-system account information.
- Exposure of server-side files may reveal usernames, paths, configuration details, secrets, source code, or other sensitive information.

- If the application can also write files, a path traversal issue may become a route to application compromise or remote code execution.

3.2.6 Defensive Measures

- Never trust client-supplied file paths or filenames.
- Resolve requested files against a fixed, canonical base directory and verify the resulting canonical path remains inside that directory.
- Prefer opaque server-side file identifiers rather than exposing filesystem paths.
- Use strict allow-lists for filenames and file types where practical.
- Run the application with least filesystem privileges and prevent access to sensitive operating-system files.

3.2.7 Learning Outcome

The exercise demonstrated the importance of tracing a parameter all the way to a filesystem operation. It also reinforced the value of inspecting both the request and raw response in Burp when a browser cannot render the returned file content directly.

Suggested classification: CWE-22: Improper Limitation of a Pathname to a Restricted Directory (Path Traversal)

3.2.8 Evidence

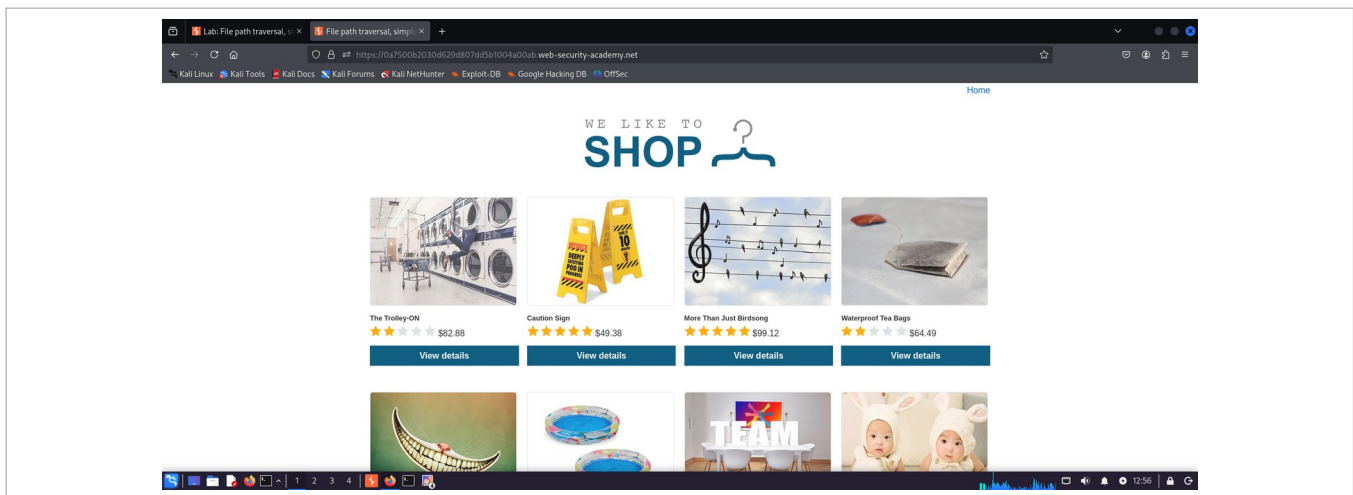
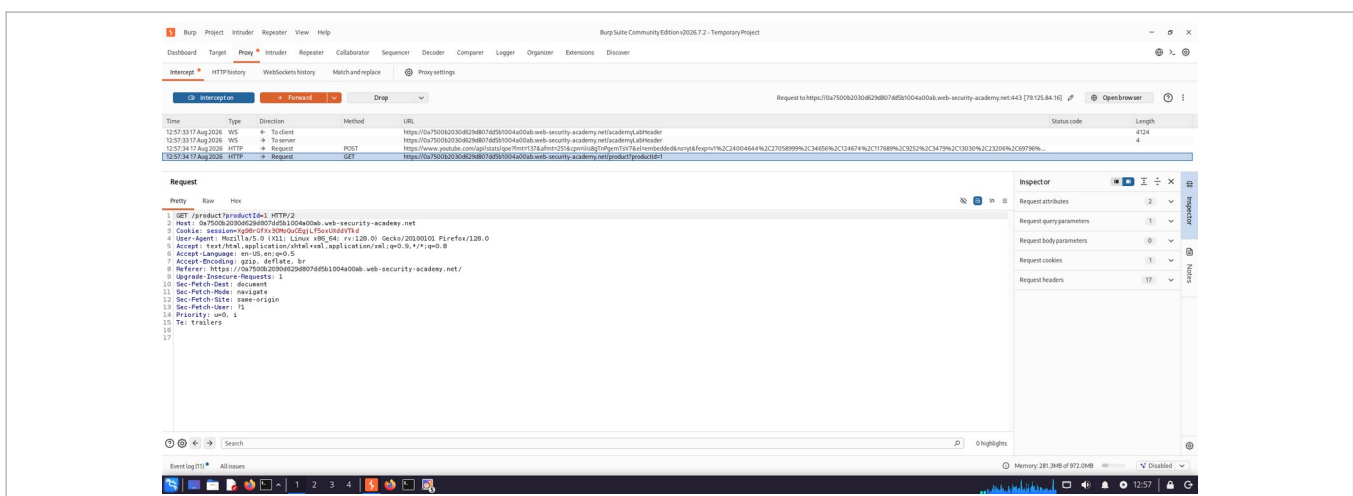


Figure 6 — Baseline application.





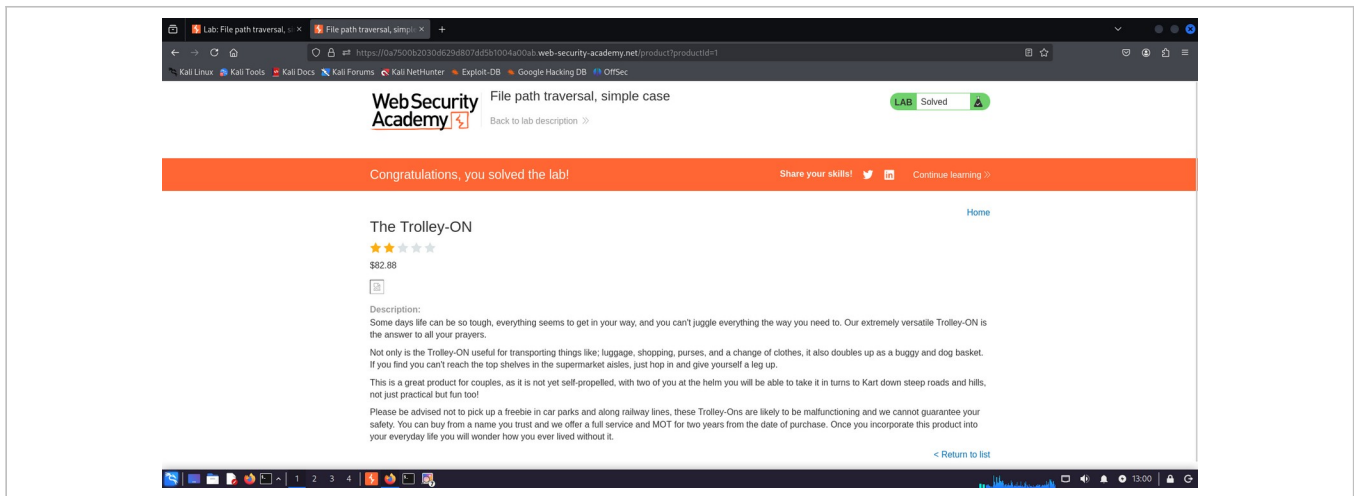


Figure 11 — PortSwigger lab completion proof.

3.3 Reflected XSS into HTML Context with Nothing Encoded

Vulnerability	Difficulty	Risk Context	Lab Status
Reflected Cross-Site Scripting (XSS)	Apprentice	High — arbitrary JavaScript execution in a victim's browser can enable session abuse, phishing, DOM manipulation, or data theft.	Completed

3.3.1 Lab Problem Statement

Official lab objective: This lab contains a simple reflected cross-site scripting vulnerability in the search functionality. To solve the lab, perform a cross-site scripting attack that calls the alert function.

Official *PortSwigger* *lab* *page:*
<https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>

3.3.2 Vulnerability Overview

Reflected XSS occurs when attacker-controlled data from an HTTP request is inserted into the immediate response in an executable browser context. Because the lab reflects the search term into HTML without encoding, markup supplied by the attacker is interpreted by the browser.

3.3.3 Assessment Objective

Demonstrate that the search parameter is reflected into the HTML response without appropriate output encoding and execute a JavaScript proof of concept.

3.3.4 Exploitation Procedure

1. Open the search functionality and submit a normal search term.
2. Capture the search request with Burp Suite and identify the search parameter.
3. Send the request to Repeater or directly test the search field with a harmless XSS proof of concept.
4. Inject `<script>alert(1)</script>` into the search parameter. The payload is reflected into the HTML response without encoding.
5. Load the resulting response in the browser and confirm JavaScript execution.
6. Capture the solved banner as the final proof of successful exploitation.

3.3.5 Impact / Security Significance

- Arbitrary JavaScript execution in the security context of the vulnerable site.
- Potential theft or manipulation of accessible application data and user actions.

- Can facilitate phishing, session compromise, malicious redirects, or chaining with other client-side vulnerabilities.

3.3.6 Defensive Measures

- Contextually encode all untrusted output before inserting it into HTML, attributes, JavaScript, CSS, or URLs.
- Prefer safe templating frameworks that escape output by default.
- Apply a strong Content Security Policy as defense in depth; do not treat CSP as a replacement for output encoding.
- Validate input where business requirements permit, but do not rely on blacklist filtering as the primary XSS defense.
- Use security-focused code review and automated XSS testing for user-controlled reflection points.

3.3.7 Learning Outcome

The lab showed how quickly a seemingly harmless search parameter can become a code-execution sink when output encoding is missing. It also reinforced the need to identify the exact reflection context before selecting a proof-of-concept payload.

Suggested classification: CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)

3.3.8 Evidence

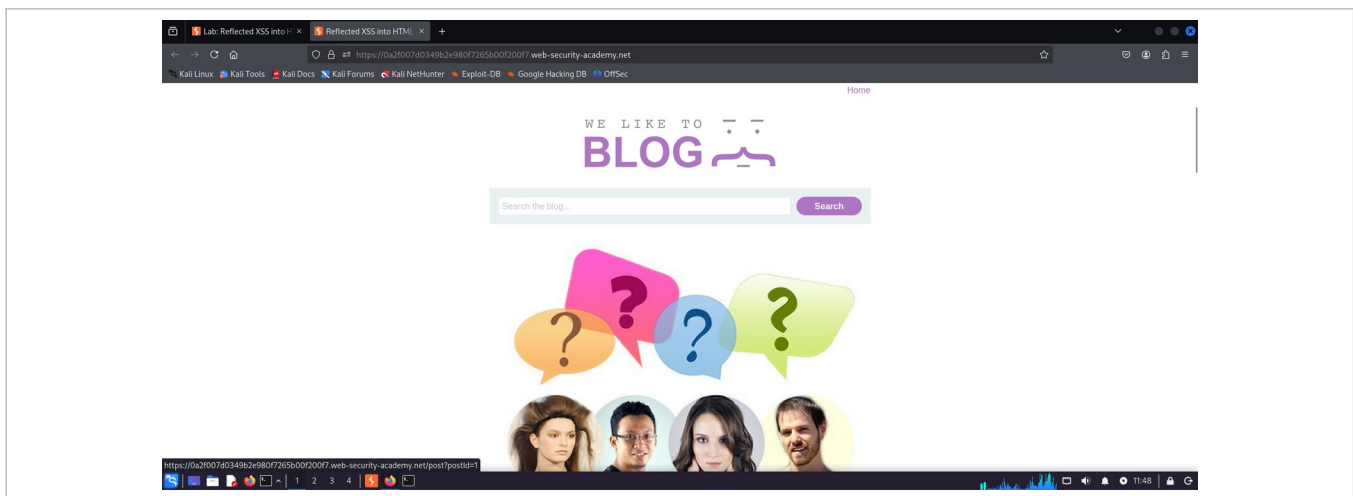


Figure 12 — Normal search functionality.

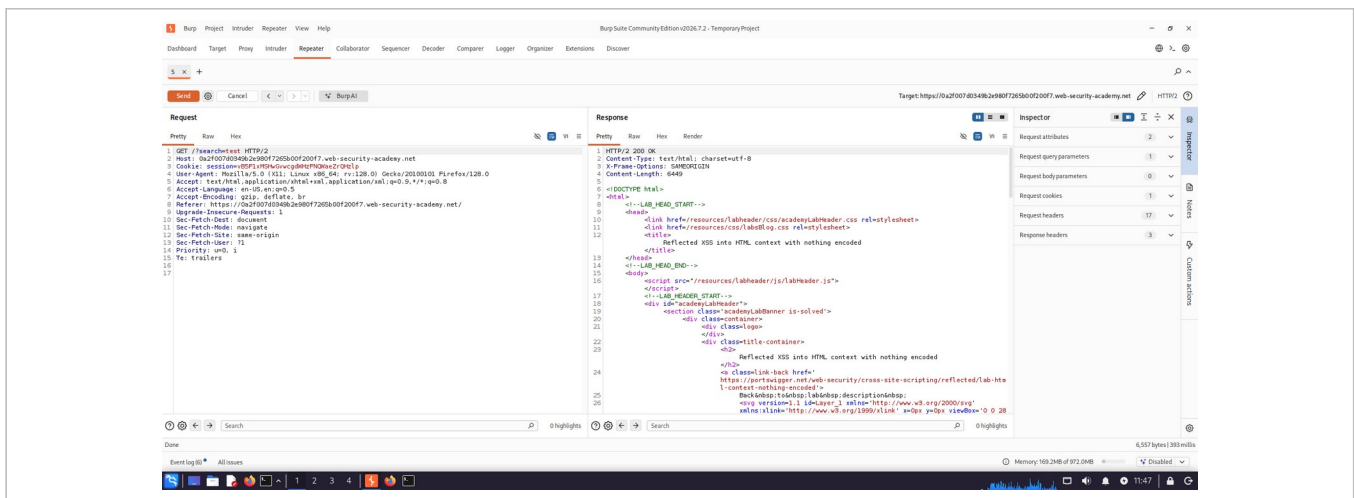


Figure 13 — Search request captured in Burp Suite.

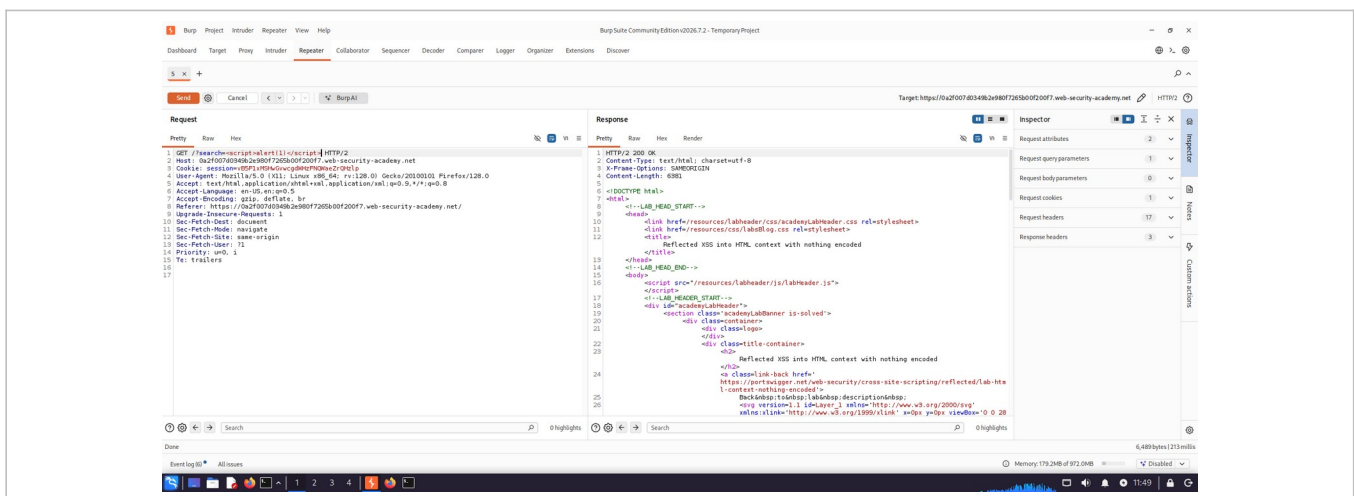


Figure 14 — Reflected XSS payload supplied through the search input.

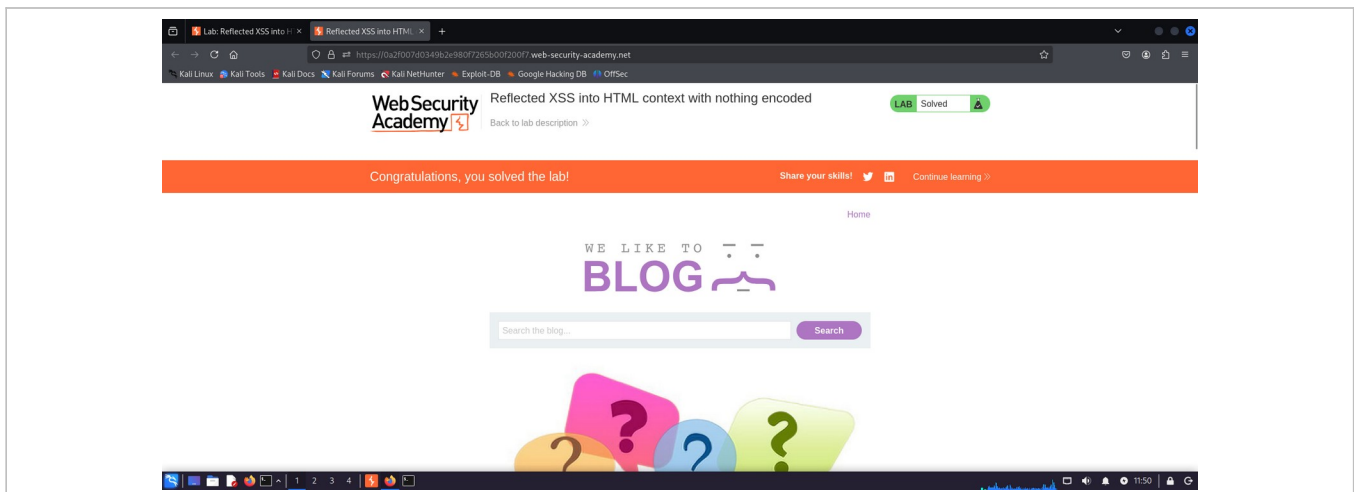


Figure 15 — PortSwigger lab completion proof.

3.4 User Role Controlled by Request Parameter

Vulnerability	Difficulty	Risk Context	Lab Status
Broken Access Control / Parameter-Based Privilege Escalation	Apprentice	Critical — a user-controlled authorization value can permit unauthorized administrative functionality.	Completed

3.4.1 Lab Problem Statement

Official lab objective: This lab has an admin panel at /admin, which identifies administrators using a forgeable cookie. Solve the lab by accessing the admin panel and using it to delete the user carlos.

Official PortSwigger lab page: <https://portswigger.net/web-security/access-control/lab-user-role-controlled-by-request-parameter>

3.4.2 Vulnerability Overview

Broken access control occurs when the server makes authorization decisions using values that an untrusted user can modify. In this lab, the application trusts an Admin=false cookie; changing that client-controlled value to Admin=true causes the server to treat the user as an administrator.

3.4.3 Assessment Objective

Escalate from a normal user to an administrative role by modifying the authorization-related cookie and then use the admin panel to delete the target user.

3.4.4 Exploitation Procedure

1. Browse to the /admin endpoint while logged in as the normal user and confirm access is denied.
2. Submit the login request through Burp Suite and inspect the response that establishes the session.
3. Identify the Admin=false cookie in the response.
4. Modify the authorization cookie from Admin=false to Admin=true before forwarding the request.
5. Request the /admin endpoint again and observe that the administrative panel is now accessible.
6. Use the administrative functionality to delete carlos and verify that the lab is solved.

3.4.5 Impact / Security Significance

- Unauthorized vertical privilege escalation from a normal user to an administrator.
- Ability to access administrative functionality and modify or delete user accounts.

- Potential compromise of all application functionality protected only by the affected authorization mechanism.

3.4.6 Defensive Measures

- Never trust role, privilege, or authorization flags supplied by the client.
- Store authorization state server-side and derive permissions from a trusted authenticated session.
- Enforce authorization checks on every sensitive endpoint and action, not only on UI navigation.
- Use deny-by-default access control and centralized authorization middleware.
- Add automated authorization tests for horizontal and vertical privilege-escalation scenarios.

3.4.7 Learning Outcome

This lab reinforced the difference between authentication and authorization. A valid login does not imply administrative privileges, and any client-controlled role indicator must be treated as untrusted input.

Suggested classification: CWE-284: Improper Access Control

3.4.8 Evidence

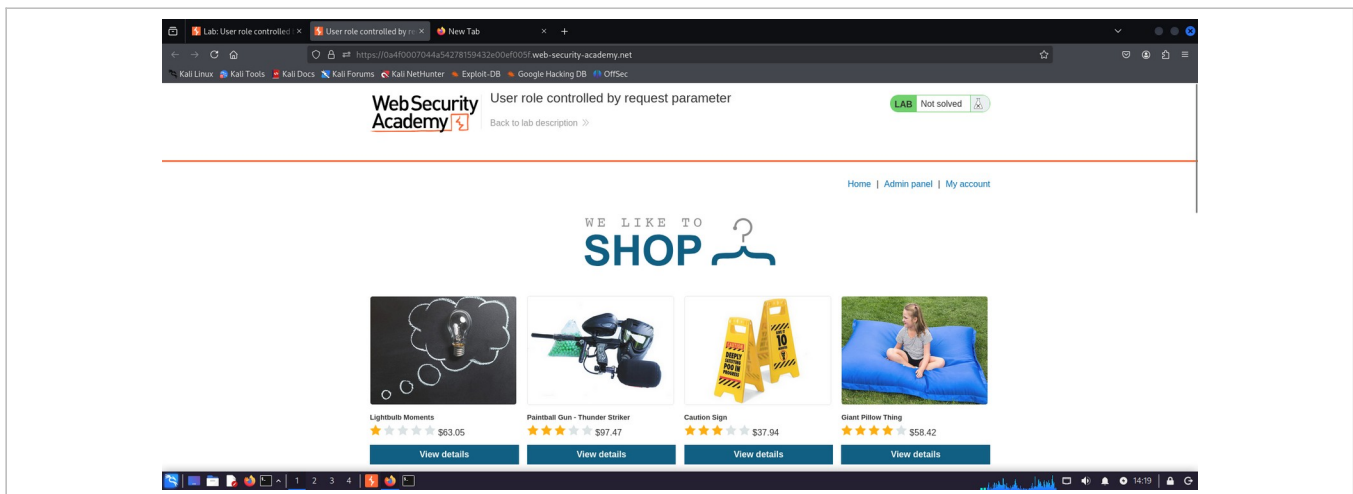
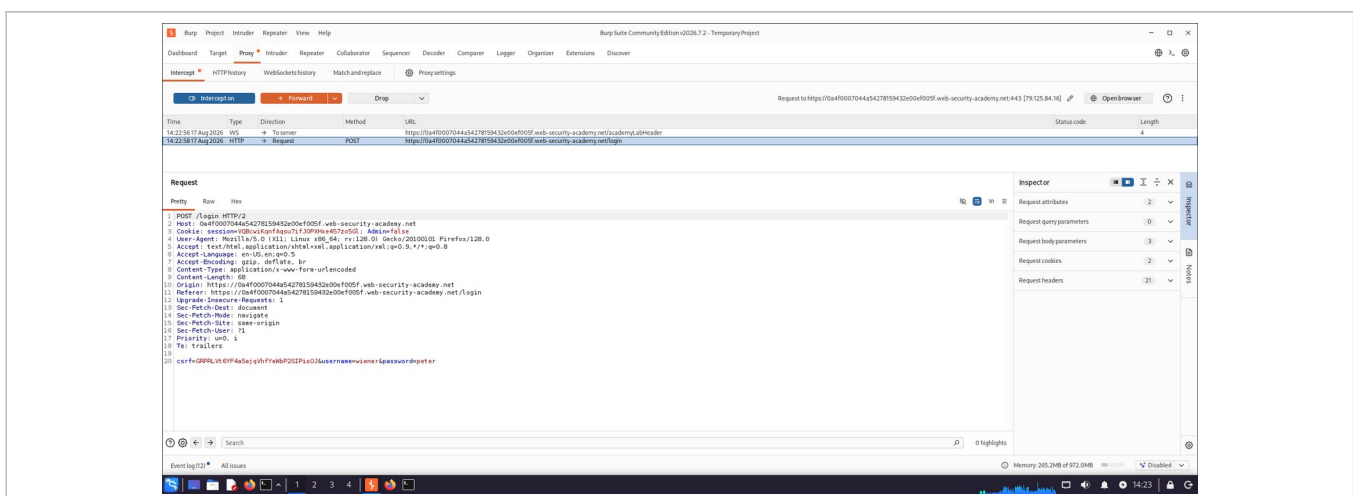


Figure 16 — Normal application state.





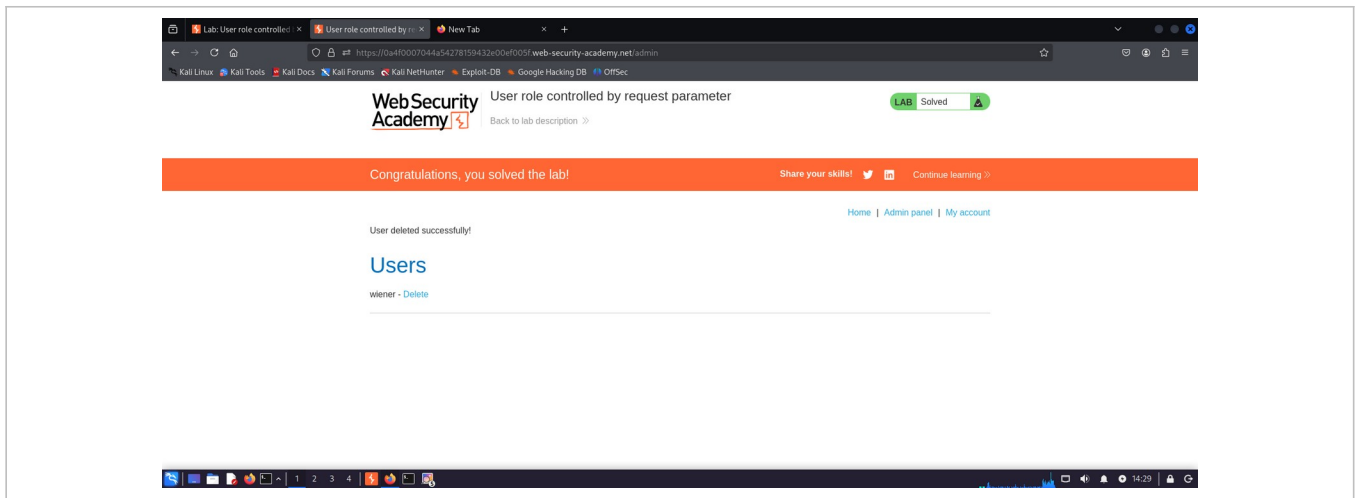


Figure 21 — PortSwigger lab completion proof.

3.5 CSRF Vulnerability with No Defenses

Vulnerability	Difficulty	Risk Context	Lab Status
Cross-Site Request Forgery (CSRF)	Apprentice	High — an attacker can cause an authenticated victim's browser to perform a state-changing action without the victim intentionally requesting it.	Completed

3.5.1 Lab Problem Statement

Official lab objective: The lab demonstrates a CSRF vulnerability in a state-changing account action with no effective anti-CSRF defense. The objective is to use the provided exploit server to host an HTML page that changes the viewer's email address.

Official PortSwigger lab page: <https://portswigger.net/web-security/csrf/lab-no-defenses>

3.5.2 Vulnerability Overview

CSRF abuses the browser's automatic inclusion of authenticated session credentials. When a sensitive endpoint accepts a cross-site request without a CSRF token or equivalent defense, an attacker can craft a page that causes a victim's browser to submit the request while the victim is logged in.

3.5.3 Assessment Objective

Construct and host a proof-of-concept page that causes the victim's authenticated browser to submit the change-email request.

3.5.4 Exploitation Procedure

1. Log in to the lab account and perform the normal Update email action.
2. Capture the resulting POST request in Burp Suite and identify the state-changing endpoint and email parameter.
3. Confirm that the request does not contain an unpredictable CSRF token or equivalent origin validation.
4. Create an HTML proof of concept on the PortSwigger exploit server that submits the change-email form automatically.
5. Test the exploit and confirm that the browser sends the authenticated request to the target application.
6. Deliver the exploit to the simulated victim and verify the lab status changes to solved.

3.5.5 Impact / Security Significance

- Unauthorized modification of a victim's account email address.

- Potential account takeover chains if the changed email is later used for password recovery or account verification.
- Other state-changing functions may be similarly exploitable if they lack CSRF defenses.

3.5.6 Defensive Measures

- Use cryptographically random, session-bound CSRF tokens on state-changing requests.
- Validate CSRF tokens server-side and reject missing, invalid, or mismatched tokens.
- Use SameSite cookie restrictions as defense in depth, especially for sensitive session cookies.
- Validate request origin using appropriate Origin/Referer checks where suitable.
- Prefer state-changing operations via POST/PUT/PATCH/DELETE rather than GET, and never rely on HTTP method alone as a CSRF defense.

3.5.7 Learning Outcome

The exercise demonstrated that CSRF is primarily an authorization-of-action problem: the browser may be correctly authenticated, yet the application fails to verify whether the requested state change was intentionally initiated by the user.

Suggested classification: CWE-352: Cross-Site Request Forgery (CSRF)

3.5.8 Evidence

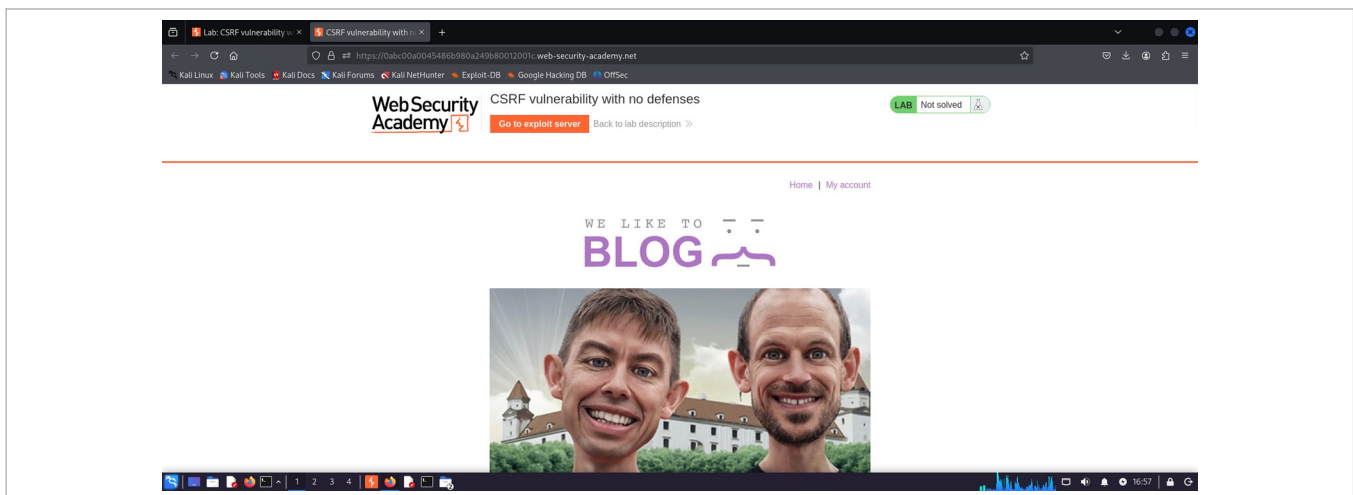


Figure 22 — Normal application/account page.

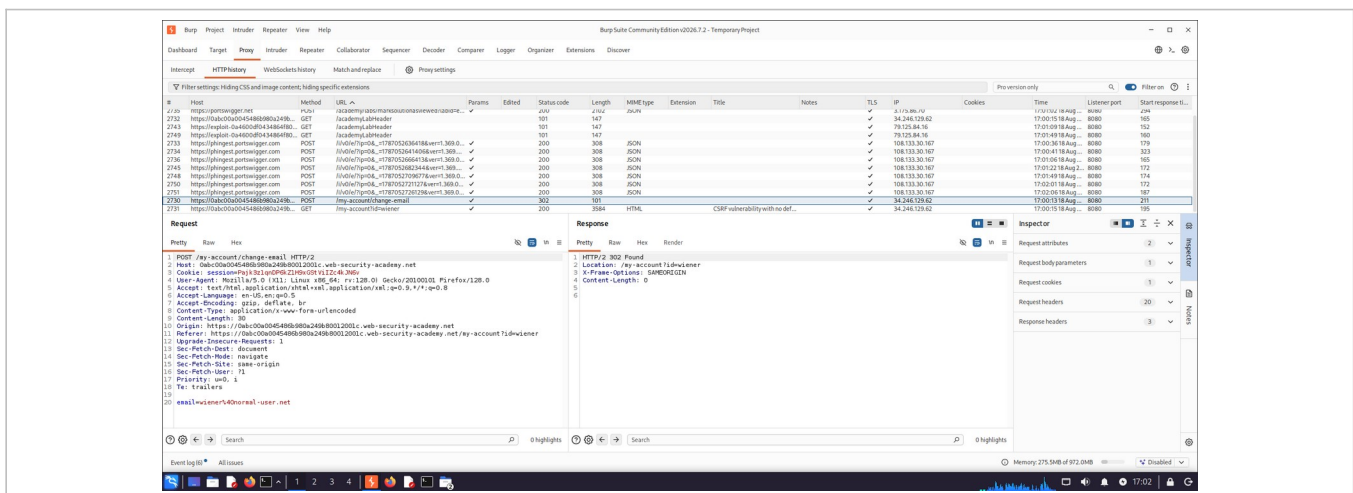


Figure 23 — Change-email request captured in Burp Suite.

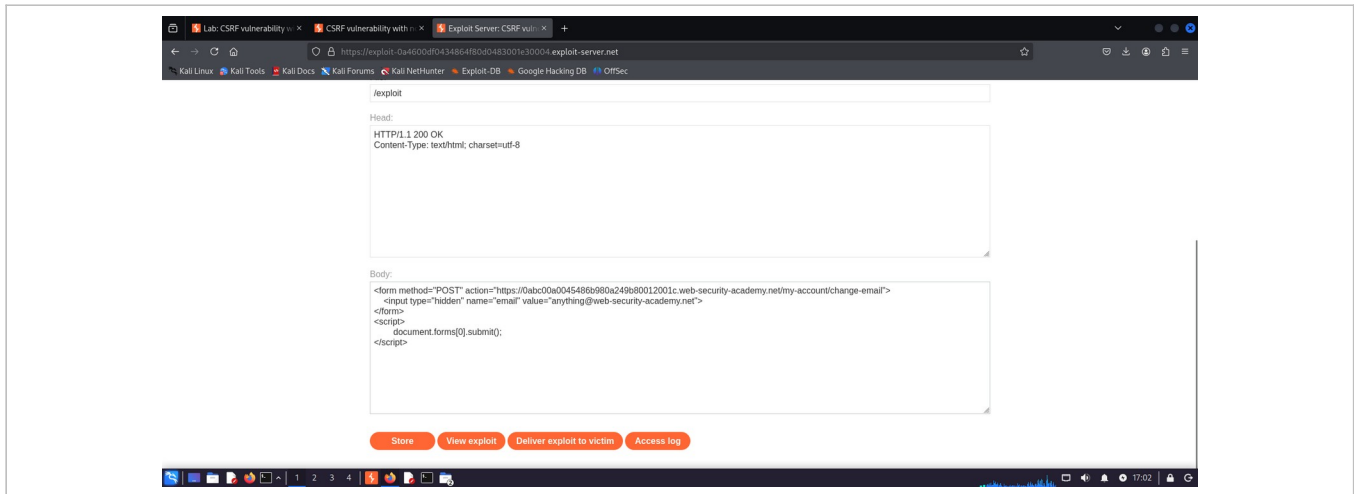


Figure 24 — CSRF proof-of-concept hosted on the exploit server.

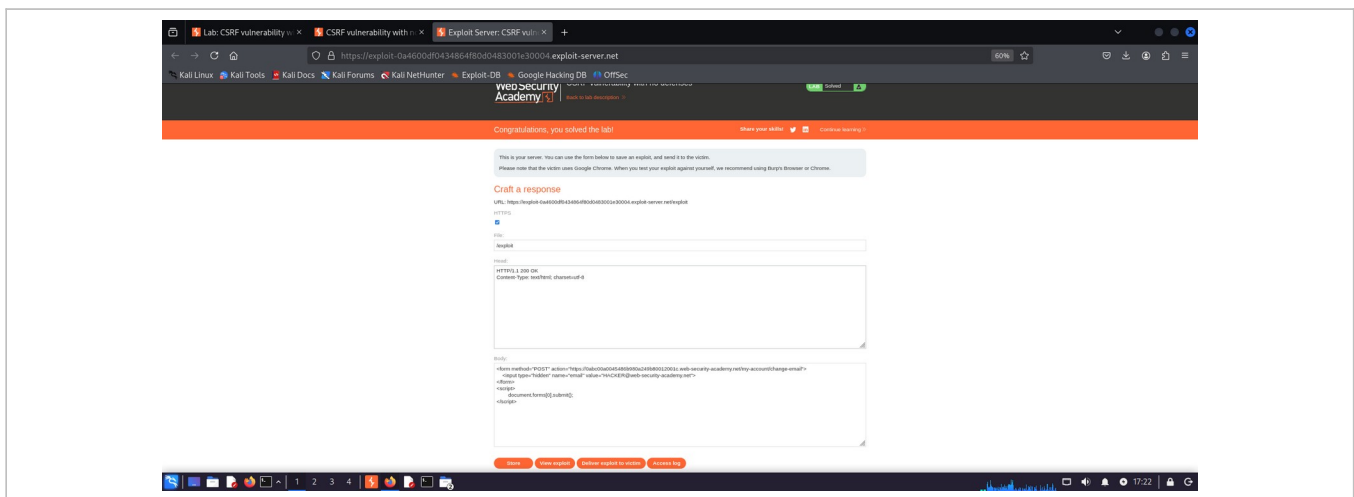


Figure 25 — PortSwigger lab completion proof.

4. Cross-Lab Security Analysis

4.1 Common Root Causes Observed

Trusting Unvalidated Input

SQLi, path traversal, and reflected XSS demonstrate how attacker-controlled input can reach sensitive interpreters or sinks.

Missing Server-Side Authorization

The access-control lab demonstrates that client-controlled role indicators cannot be trusted for privilege decisions.

Missing Request-Intent Validation

The CSRF lab shows that authenticated requests require protection against unauthorized cross-site initiation.

Insufficient Boundary Validation

Each lab crosses a boundary — database, filesystem, browser DOM, authorization layer, or session state — where the application failed to enforce a secure constraint.

4.2 Comparison of Testing Techniques

Lab	Primary Input	Burp Technique	Success Indicator
SQLi	category	Proxy + Repeater	Unreleased products returned
Path Traversal	filename	Proxy + Repeater	/etc/passwd content returned
Reflected XSS	search	Proxy + browser	JavaScript executes
Broken Access Control	Admin cookie	Proxy + Repeater	Admin panel accessible
CSRF	change-email request	Proxy + exploit server	Victim state changed

4.3 Defensive Themes

- Validate security decisions on the server, not in client-controlled fields.
- Use safe APIs and frameworks that separate data from executable syntax.
- Apply contextual output encoding for browser-facing data.
- Constrain filesystem operations to canonical, authorized directories.
- Use least privilege for database and operating-system service accounts.
- Protect state-changing requests with CSRF tokens and appropriate cookie/origin controls.
- Continuously test authorization and input-handling logic with negative and abuse-case test cases.

5. Learning Outcomes

- Improved practical understanding of how web applications process HTTP requests and parameters.
- Developed hands-on experience using Burp Suite to intercept, modify, replay, and analyze requests.
- Learned to distinguish authentication failures from authorization failures.
- Understood the difference between reflected XSS and server-side input vulnerabilities such as SQL injection and path traversal.
- Learned how CSRF exploits the browser's authenticated state rather than directly bypassing authentication.
- Improved evidence-driven reporting by documenting baseline behavior, exploitation, impact, remediation, and completion proof.
- Strengthened awareness of secure-development controls including parameterized queries, output encoding, canonical path validation, server-side authorization, and CSRF protection.

6. Conclusion

The completed labs demonstrate successful completion of at least five Apprentice-level PortSwigger Web Security Academy labs. The primary five labs selected for this report provide broad coverage across database security, filesystem security, client-side security, authorization, and session/request integrity. Each lab was validated using practical exploitation evidence and a PortSwigger completion proof.

The central lesson across the exercises is that secure web applications must treat all client-controlled data as untrusted and enforce security decisions at the appropriate server-side trust boundary. Proper input handling, output encoding, authorization enforcement, least privilege, and request-integrity controls should be implemented together rather than relying on obscurity or client-side checks.

Appendix A — Additional Completed Lab Evidence

The evidence archive contains proof for additional Apprentice-level exercises beyond the five primary labs required by the project. These are recorded here as supplementary practice/completion evidence.

Category	Lab	Evidence
SQL Injection	SQL injection vulnerability allowing login bypass	Evidence: L02-E01 through L02-E03
XSS	Stored XSS into HTML context with nothing encoded	Evidence: L02-E01 through L02-E03
DOM XSS	DOM XSS in document.write sink using source location.search	Evidence: L03-E01 through L03-E03
DOM XSS	DOM XSS in innerHTML sink using source location.search	Evidence: L04-E01 through L04-E04
DOM XSS / jQuery	DOM XSS in jQuery anchor href attribute sink using location.search source	Evidence: L05-E01 through L05-E04
Access Control	Unprotected admin functionality	Evidence: L01-E01 through L01-E04
Access Control	Unprotected admin functionality with unpredictable URL	Evidence: L02-E01 through L02-E04
Access Control	User role can be modified in user profile	Evidence: L04-E01 through L04-E07
Access Control	User ID controlled by request parameter, with unpredictable user IDs	Evidence: L05-E01 through L05-E04

Appendix B — Evidence Inventory

The original evidence archives were preserved as the source material for this report. The filenames were used to maintain traceability between the report captions and the collected screenshots.

- 01-Lab-SQL-Injection-Hidden-Data.zip
- 02-Lab-Path-Traversal.zip
- 03-Lab-Reflected-XSS.zip
- 04-Lab-Broken-Access-Control.zip
- 05-Lab-CSRF.zip

Appendix C — Official PortSwigger References

1. SQL Injection — hidden data lab: <https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>
2. Path Traversal — simple case lab:
<https://portswigger.net/web-security/file-path-traversal/lab-simple>
3. Reflected XSS — HTML context, nothing encoded:
<https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>
4. Access Control — user role controlled by request parameter: <https://portswigger.net/web-security/access-control/lab-user-role-controlled-by-request-parameter>
5. CSRF — no defenses: <https://portswigger.net/web-security/csrf/lab-no-defenses>
6. PortSwigger Web Security Academy: <https://portswigger.net/web-security>

Yashdeep Sankhla

Cyber Security Intern — CyberSNova